ntic master
revolucionamos la comunicación

TECNOLOGÍAS DEL BIG DATA

TECNOLOGÍAS DE SERVICIO

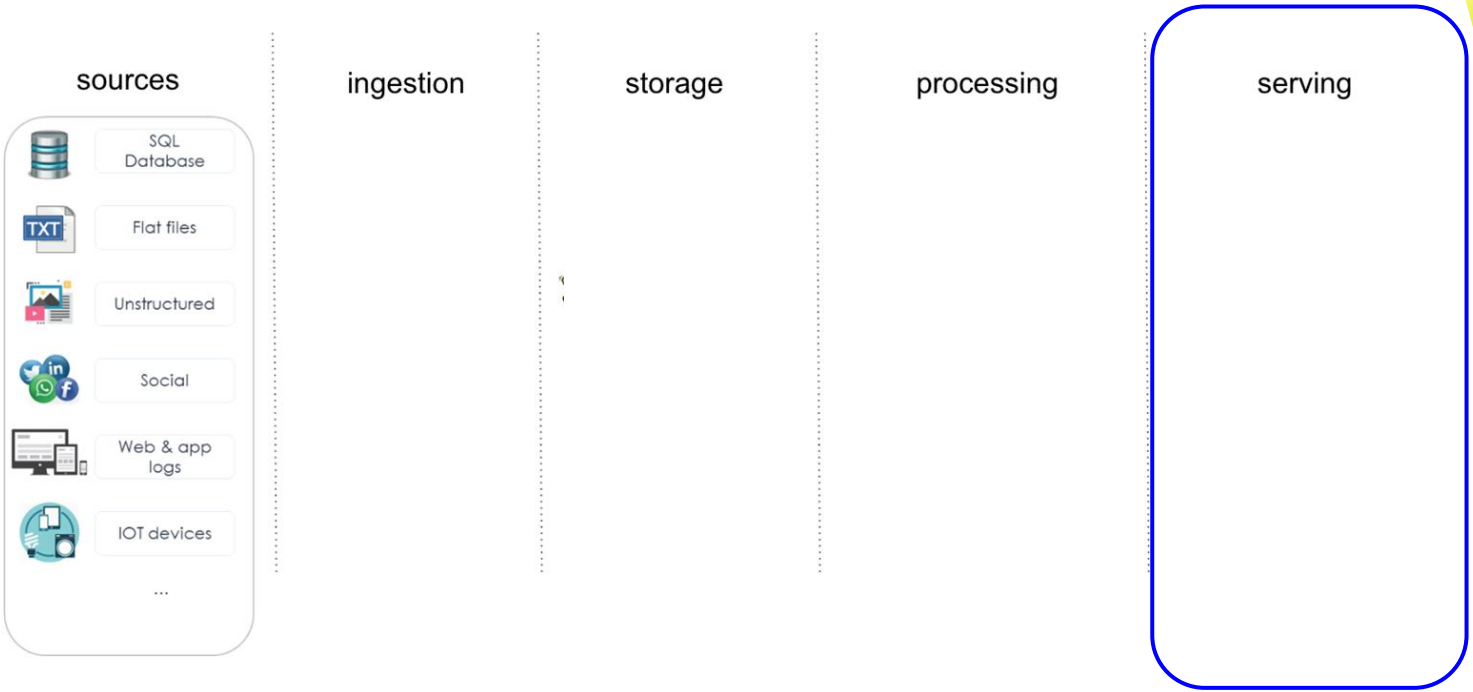
Agenda

- Definición
- Teorema CAP
- NoSQL
  - Key-Value Stores
  - Column Family Stores
  - Document Stores
  - Graph Stores
  - Time Series Stores
  - Search Engines
- Resumen

1.

DEFINICIÓN

Definición


sources
ingestion
storage
processing
serving
SQL Database
Flat files
Unstructured
Social
Web & app logs
IOT devices
...

Servicio de Datos

El servicio de datos trata de **disponibilizar los resultados del procesamiento para que puedan ser consumidos** de la manera requerida por los usuarios o aplicaciones finales.

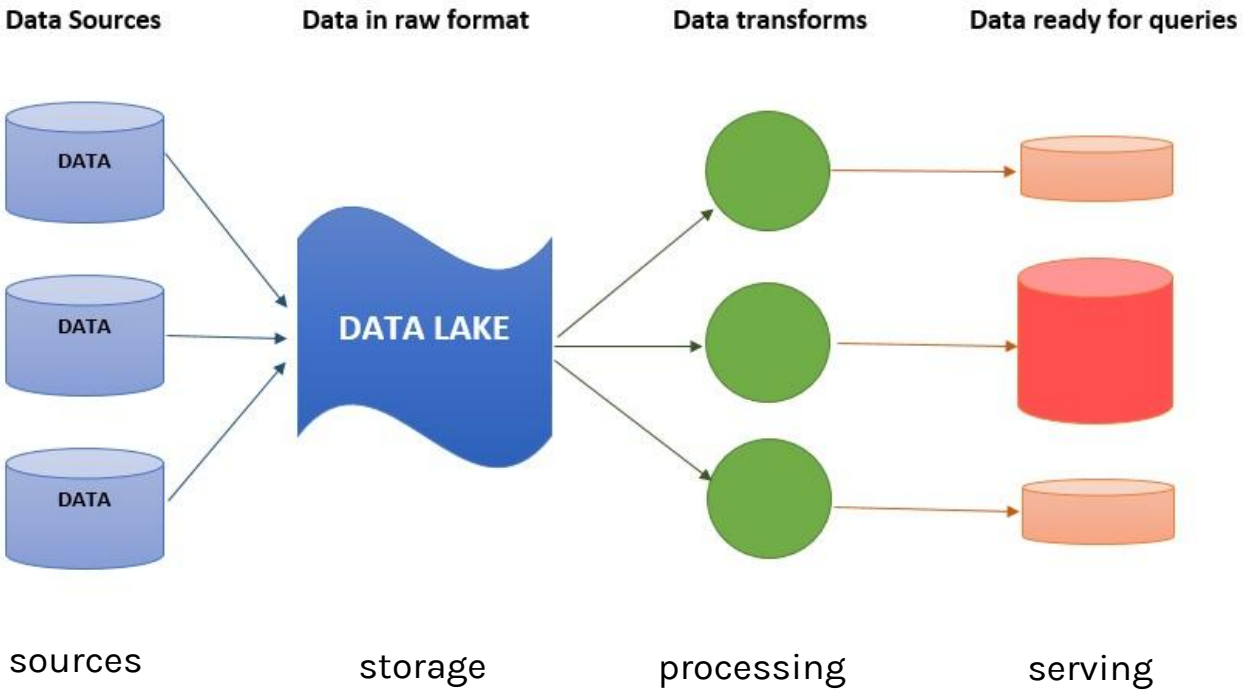
Servicio de Datos

Esta capa suele estar compuesta por **múltiples tipos de tecnologías** de servicio de datos.

La razón es que diferentes patrones de consumo de datos requieren diferentes tecnologías

No existe una tecnología capaz de cubrir todas las necesidades

Servicio de Datos



Servicio de Datos - Estructurados

¿Cómo servimos **datos estructurados**?

Servicio de Datos - Estructurados

Si es **small data** podemos disponibilizarlos en **bases de datos relacionales (SQL)**



Servicio de Datos - Estructurados

Si es **big data** podemos disponibilizarlos en **datastores NoSQL** o directamente desde el **almacenamiento batch** usando una tecnología de **procesamiento** como interfaz

- **NoSQL**
- **Data Lake / Data Lakehouse**

Servicio de Datos - Semi Estructurados

¿Cómo servimos **datos semi-estructurados**?

Servicio de Datos - Semi Estructurados

Independientemente de si son **small data** o **big data** podemos disponibilizarlos en **datastores NoSQL** o directamente desde el **almacenamiento batch** usando una tecnología de **procesamiento** como interfaz

- **NoSQL**
- **Data Lake / Data Lakehouse**

Servicio de Datos - No Estructurados

¿Cómo servimos **datos no estructurados**?

Servicio de Datos - No Estructurados

Independientemente que estemos tratando con **small data** o **big data** directamente desde el **almacenamiento batch**

- Data Lake / Data Lakehouse

Servicio de Datos - Streaming

¿Cómo servimos **datos streaming**?

Servicio de Datos - Streaming

Haremos uso de tecnologías de **almacenamiento streaming.**





2.

TEOREMA CAP

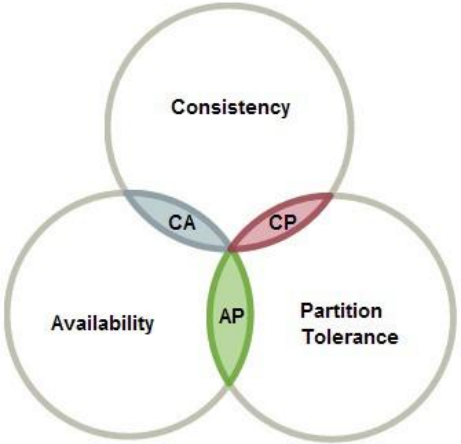
Teorema CAP

Ninguna **tecnología de almacenamiento** de datos **distribuida** puede proveer **simultáneamente** :

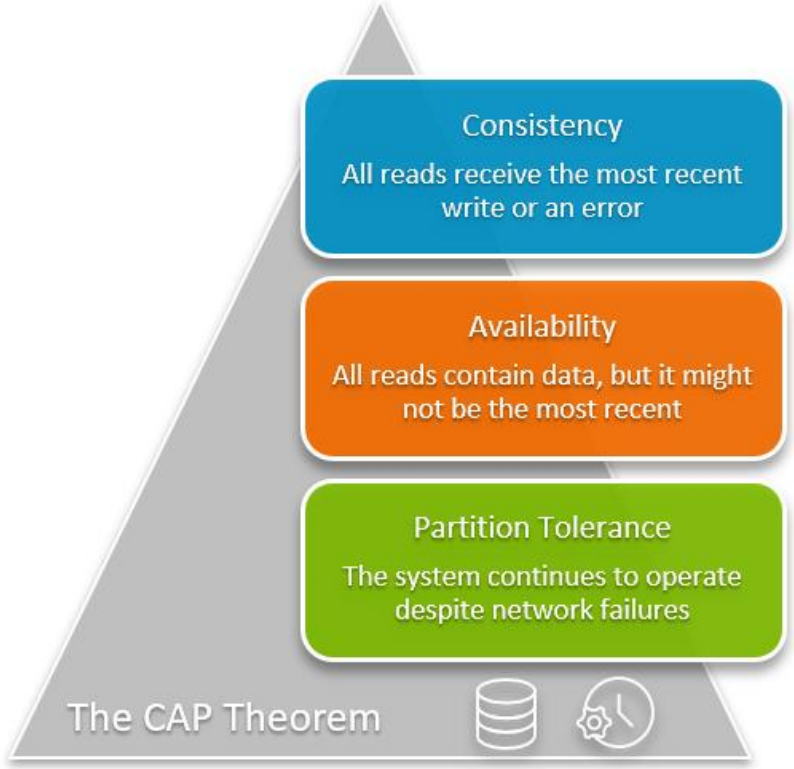
Consistencia

Disponibilidad

Tolerancia particionado



Teorema CAP


Consistency
All reads receive the most recent write or an error
Availability
All reads contain data, but it might not be the most recent
Partition Tolerance
The system continues to operate despite network failures
The CAP Theorem

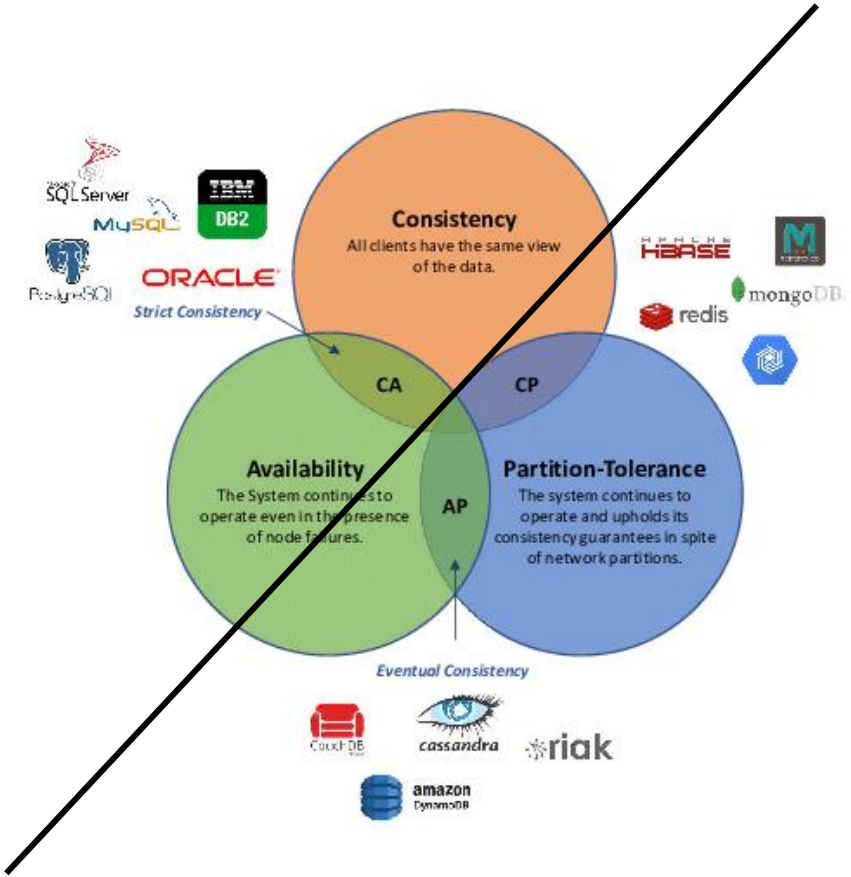
Teorema CAP

Puesto que las redes de comunicaciones no son fiables, **todo sistema distribuido debe tolerar siempre particionado de red**

Los datastores NoSQL deben elegir pues, entre proveer consistencia o proveer disponibilidad (ya que es imposible tener las tres)

Teorema CAP

SQL



NoSQL

Teorema CAP

Un datastore NoSQL provee:

Consistencia + Tolerancia Particionado (CP)

o

Disponibilidad + Tolerancia Particionado (AP)

Teorema CAP

Consistencia, Tolerancia Particionado (CP):

Sacrifica la disponibilidad para mantener los datos consistentes entre todos los nodos del cluster.

Para garantizar la consistencia parte de los datos pueden llegar a no estar disponibles o accesibles mientras dure el proceso de sincronización.

Teorema CAP

Disponibilidad, Tolerancia Particionado (AP):

Sacrifica la consistencia (con el riesgo de devolver datos obsoletos) para mejorar la disponibilidad.

Para garantizar la disponibilidad es posible que devuelva datos inconsistentes

¿Cómo **elegir** el datastore NoSQL apropiado?

Teorema CAP

Data stores que proveen **consistencia** se deben usar cuando el valor de la **información** reside en que los datos sean **precisos**.

Ejemplo: balance de cuentas bancarias

Teorema CAP

Data stores que proveen **disponibilidad** se deben usar cuando el **servicio es más importante** que la **información**.

Ejemplo: buscadores e-commerce

3.

NoSQL

NoSQL

Los datastores **NoSQL** tienen varias **ventajas** sobre las **base de datos** relacionales:

- Escalabilidad
- Modelo de datos flexible
- Especializaciones

NoSQL

Escalabilidad:

Los datastores NoSQL **escalan horizontalmente** (scale out), añadiendo más nodos a medida que el sistema necesita crecer.

NoSQL

Modelo de datos flexible:

Los datastores NoSQL abordan los problemas de los modelos de datos relacionales soportando modelos de datos **semi-estructurados** o incluso modelos en **grafo**

NoSQL

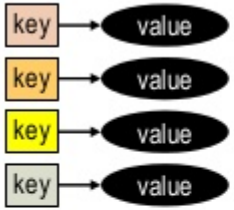
Especializaciones:

- Key-Value stores
- Column Family stores
- Document stores
- Graph stores

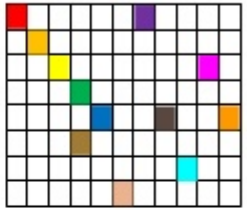


NoSQL

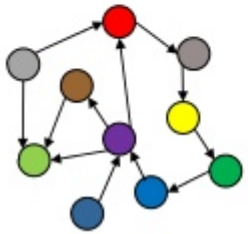
Key-Value



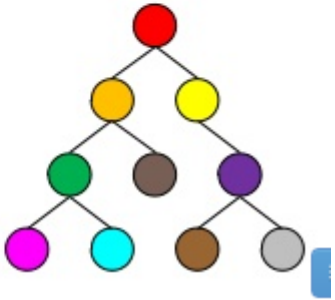
Column-Family



Graph



Document



NoSQL

Subespecializaciones adicionales:

- Time Series stores
- Search Engines

Normalmente estas subespecializaciones se implementan internamente mediante el uso de una o más de las cuatro familias anteriores.

3.1

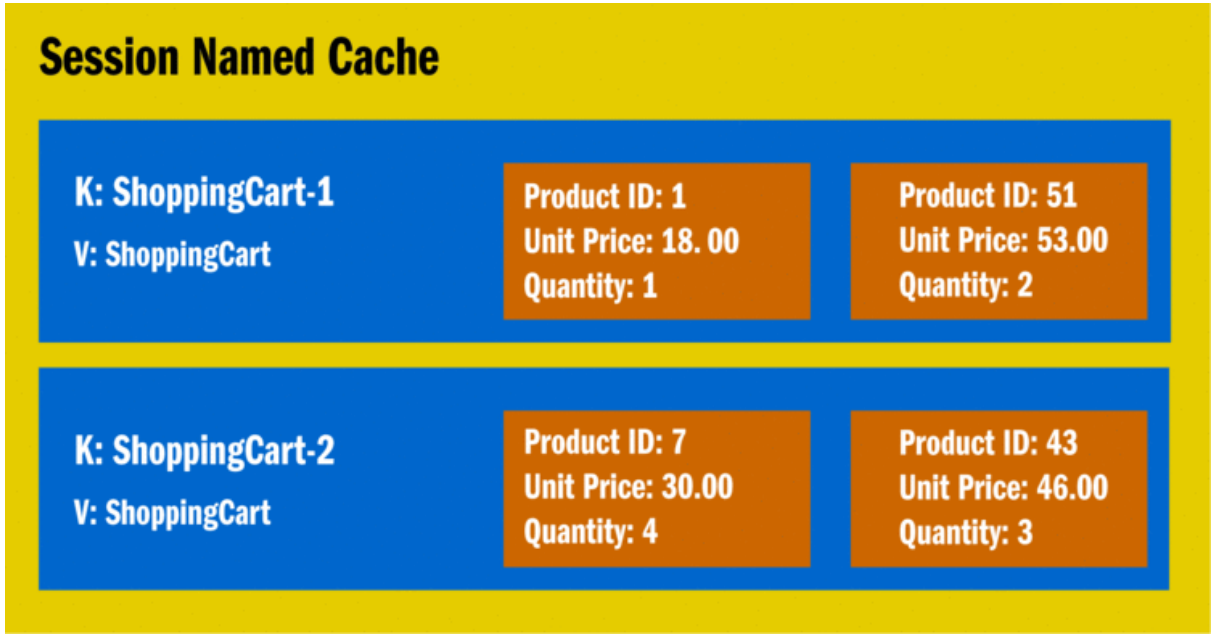
KEY-VALUE STORES

Key-Value Stores

Key-value datastores están diseñados para **acceso a los datos de baja latencia.**

Estos datastores básicamente funcionan como un diccionario/mapa distribuido mapeando una clave y un valor para proveer **lecturas y escrituras a gran velocidad**.

Key-Value Stores



Key-Value Stores

Pros

- Memoria
- Acceso ultra rápido

Contras

- No permiten consultas.
- Datos almacenados sin esquema.

Key-Value Stores

Casos de uso:

- Cacheo de datos
- Seguimiento de datos temporales en aplicaciones web. Ej: carrito compra

Tecnologías Open Source









Tecnologías Cloud







3.2

COLUMN FAMILY STORES

Column Family Stores

Los datastores columnares están diseñados para:

- **grandes volumenes datos**
- **alto rendimiento de lecturas y escrituras**

Column Family Stores

CustomerID	Column Family: Identity
001	**First name:** Mu Bae **Last name:** Min
002	**First name:** Francisco **Last name:** Vila Nova **Suffix:** Jr.
003	**First name:** Lena **Last name:** Adamcyz **Title:** Dr.

CustomerID	Column Family: Contact Info
001	**Phone number:** 555-0100 **Email:** someone@example.com
002	**Email:** vilanova@contoso.com
003	**Phone number:** 555-0120

Column Family Stores

Pros

- Mejores capacidades de consulta que key-value stores
- Alto rendimiento lectura/escritura

Cons

- Difícil de hacer consultas complejas

Column Family Stores

Casos de uso:

- Security analytics usando trafico de red
- Bioinformática con datos genéticos.
- Cotizaciones acciones.

Tecnologías Open Source









Tecnologías Cloud



Google BigTable



Azure Cosmos DB



3.3

DOCUMENT STORES

Document Stores

Los datastores basados en documentos están diseñados para:

Modelo de datos flexible

(almacenar y recuperar datos semi-estructurados)

Permiten almacenar atributos variantes en grandes datasets

Document Stores

Key	Document
1001	<pre>{ "CustomerID": 99, "OrderItems": [{ "ProductID": 2010, "Quantity": 2, "Cost": 520 }, { "ProductID": 4365, "Quantity": 1, "Cost": 18 }], "OrderDate": "04/01/2017" }</pre>
1002	<pre>{ "CustomerID": 220, "OrderItems": [{ "ProductID": 1285, "Quantity": 1, "Cost": 120 }], "OrderDate": "05/08/2017" }</pre>

Document Stores

Pros

- Toleran datos incompletos
- Más sencillo hacer consultas complejas

Cons

- Rendimiento de las consultas

Document Stores

Casos de uso

- Gestión de entidades con atributos variables como productos en un ecommerce
- Aplicaciones que usen estructuras de datos JSON

Tecnologías Open Source







elasticsearch



Tecnologías Cloud



Google Firestore



Amazon DynamoDB





Azure Cosmos DB

3.4

GRAPH STORES

Graph Stores

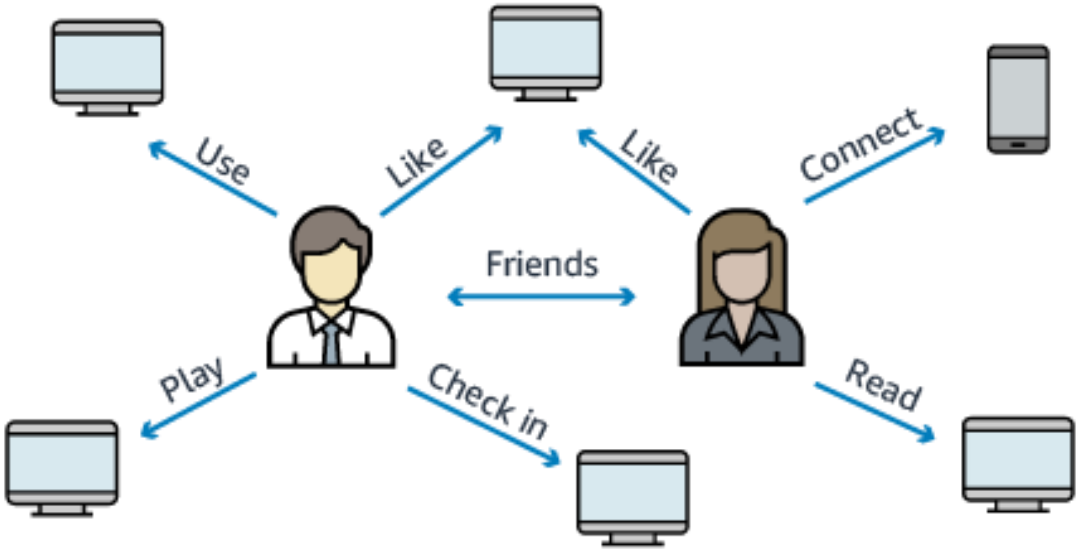
Estos datastores están diseñados para **almacenar y consultar grafos.**

Para consultar un **grafo** no podemos usar SQL.

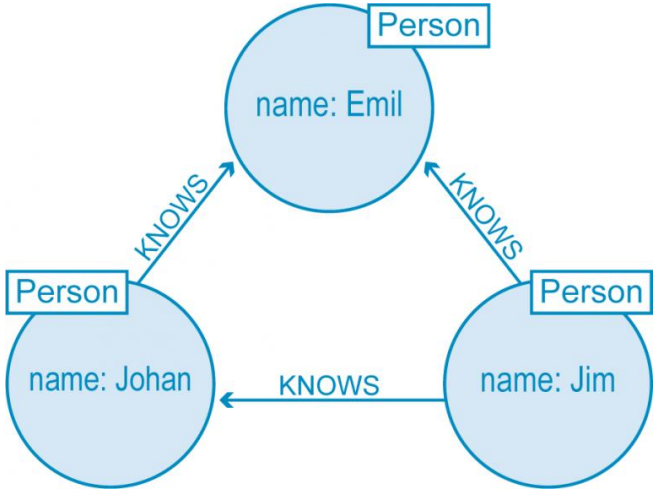
Estas tecnologías suelen proveer **su propio lenguaje de consulta**

Ejemplo: el lenguaje de consulta de Neo4j se llama Cypher

Graph Stores



Graph Stores




```
MATCH (a:Person {name:'Jim'})-[:KNOWS]->(b)-[:KNOWS]->(c),
      (a)-[:KNOWS]->(c)
RETURN b, c
```

Graph Stores

Pros

- Algoritmos de grafos
- Consultas de grafos

Cons

- Dificil de distribuir los datos de forma eficiente

Graph Stores

Casos de uso:

- Consultas en rrss : "friends of my friends"
- Reconocimiento de patrones
- Sistemas de recomendación

Tecnologías Open Source











Tecnologías Cloud



Azure Cosmos DB



Amazon Neptune

3.5

TIME SERIES STORES

Time Series Stores

Los Time Series datastores están **diseñados** para almacenar y consultar **series temporales** con grandes volúmenes de transacciones

Cada registro de datos está asociado con un instante temporal

Time Series Stores

timestamp	deviceid	value
2017-01-05T08:00:00.123	1	90.0
2017-01-05T08:00:01.225	2	75.0
2017-01-05T08:01:01.525	2	78.0

Time Series Stores

Casos de uso:

- IoT
- Analítica en tiempo real

Tecnologías Open Source



influxdb



Prometheus

graphite

3.6

SEARCH ENGINES

Search Engines

Los motores de búsqueda están **diseñados** para **la búsqueda textual:**

búsqueda != consulta

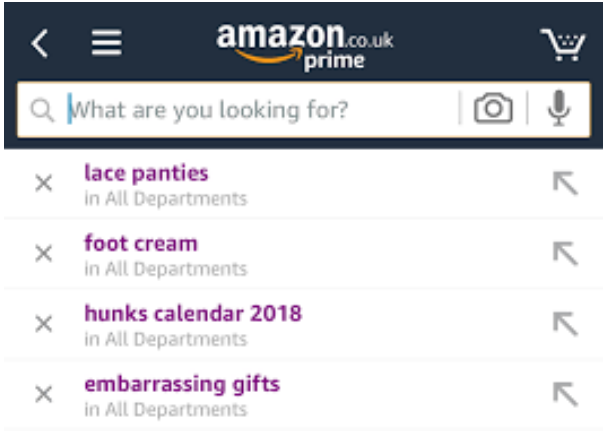
Search Engines

Cuando hacemos una consulta a un datastore o base de datos necesitamos conocer el modelo o las estructuras de datos de antemano.

Cuando buscamos solo necesitamos saber qué buscar (keywords).

Search Engines





Search Engines

Pros

- Soportan expresiones complejas de búsqueda

- Búsqueda full text

Search Engines

Casos de uso:

- Aplicaciones de centralización de logs
- Ecommerce autocompleter

Search Engines






hadoop
HDFS
teradata.
IBM
DB2
Microsoft
SQL Server
MariaDB
APACHE
HBASE
Amazon RDS
SQL
ORACLE
DATABASE
MySQL
druid
SQL
snowflake
PostgreSQL
APACHE
kafka
redis
SAP S/4 HANA
PULSAR
mongoDB
CouchDB
CLOUD BIGTABLE
Azure Cosmos DB
elasticsearch
Apache
Solr
MEMCACHED
neo4j
ArangoDB
OrientDB
cassandra
amazon
DynamoDB